

2.6 How to Decide Which Algorithm to Use

الگورتھم چننے سے پہلے دیکھیں کہ ڈیٹا کتنا بڑا ہے، ترتیب شدہ ہے یا نہیں، اور ہمیں رفتار چاہیے یا میموری کی بچت۔

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 11 of 11

2.6 How to Decide Which Algorithm to Use

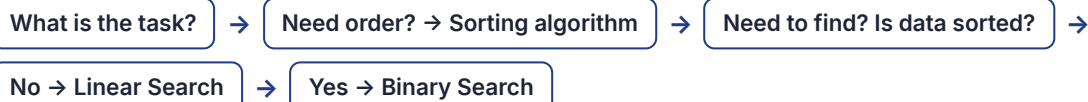
LEARN IT

Before choosing an algorithm, ask three questions:

- **What is the input size?** (10 items or 10 million?)
- **Is the data sorted?**
- **Do we need speed or memory efficiency?**

Simple decision guide:

- If searching on **unsorted** data → use **Linear Search**.
- If searching on **sorted** data → use **Binary Search**.
- If data needs **ordering** → use a **sorting** algorithm.



Comparison of the four algorithms

Algorithm	Type	Best case	Worst case	Extra memory	Needs sorted data?	Stable?
Bubble Sort	Sorting	$O(N^2)$ as written; $O(N)$ with an early-stop check	$O(N^2)$	None — $O(1)$	—	Yes
Selection Sort	Sorting	$O(N^2)$	$O(N^2)$	None — $O(1)$	—	No
Linear Search	Searching	$O(1)$ (first item)	$O(N)$ (last item / absent)	None — $O(1)$	No	—
Binary Search	Searching	$O(1)$ (item at first MID)	$O(\log N)$	None — $O(1)$	Yes	—

Point	Bubble Sort	Selection Sort
What is compared	Adjacent elements (j and j+1)	Fixed position K with every later element
Inner loop starts at	0	outer + 1
After each pass	Largest moves to the end	Smallest moves to the front
Swaps	Many	Fewer
Stable	Yes	No

Point	Linear Search	Binary Search
Method	Checks one by one	Splits the list in half each step
Data required	Sorted or unsorted	Sorted only
Worst case (1M items)	1,000,000 checks	About 20 checks
Code	Very simple	More complex, off-by-one risk
Best for	Small or unsorted lists, one-off searches	Large, sorted lists searched many times

WORKED EXAMPLE

Choosing in practice

- **A shopkeeper's list of 15 items, written in random order, searched once:** Linear Search — the list is unsorted and tiny; sorting first would waste time.
- **NADRA looking up a CNIC number among millions of sorted records, thousands of times a day:** Binary Search — data is sorted, huge, and searched repeatedly.
- **A teacher arranging 40 students' marks into a merit list:** a sorting algorithm; for 40 items even Bubble or Selection Sort is acceptable.
- **An unsorted list of 1 million records that will be searched thousands of times:** sort it once, then use Binary Search for every search.

REMEMBER IT

Two keywords decide most exam answers: "**sorted?**" and "**how big?**". Unsorted or small → Linear. Sorted and big → Binary.

COMMON MISTAKE

"Binary Search is always better" is wrong. On unsorted data it does not work at all, and for a single search on a small unsorted list, sorting first costs more than a simple Linear Search.

اردو میں سمجھیں

الگورتھم چننے سے پہلے دیکھیں کہ ڈیٹا کتنا بڑا ہے، ترتیب شدہ ہے یا نہیں، اور ہمیں رفتار چاہیے یا میموری کی بچت۔ غیر ترتیب شدہ ڈیٹا میں تلاش کے لیے لینیئر سرچ، ترتیب شدہ ڈیٹا میں تلاش کے لیے بائنری سرچ، اور ڈیٹا کو ترتیب دینے کے لیے سارٹنگ الگورتھم استعمال کریں۔

CHECK YOURSELF

1. Which search would you use on an unsorted list of phone contacts?
2. What is the worst-case time complexity of Binary Search?
3. Which sort has the same time complexity in its best and worst case?

Answers: 1. Linear Search. 2. $O(\log N)$. 3. Selection Sort ($O(N^2)$ in both).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. If the list is unsorted and n is small, search with:

- a. Linear search
- b. Binary search immediately
- c. Dijkstra
- d. Quicksort only

MCQ 2. If the list is sorted and n is huge, search with:

- a. Binary search
- b. Linear only
- c. Bubble
- d. OSI

MCQ 3. If you must sort a class of 20 names by hand:

- a. Simple $O(n^2)$ is fine
- b. You must use $O(n \log n)$ hardware
- c. K-maps
- d. HCI

MCQ 4. Memory is tiny: prefer:

- a. In-place algorithms
- b. Huge extra arrays
- c. Storing every trace forever
- d. Videos

MCQ 5. Need a stable sort of equal keys:

- a. Do not pick unstable selection without care
- b. Always XOR
- c. Always pie
- d. Always Agile

MCQ 6. The first questions are:

- a. n ? sorted? time vs memory?
- b. Font? Colour? OSI?
- c. NAND count only
- d. Figma frames

Extra short questions

1. List four questions before choosing an algorithm.
2. When is $O(n^2)$ OK?
3. When must you use binary search?
4. Give a wrong choice and why.

Extra long question

1. A college has 30 students (daily) and 50,000 archived papers (yearly search). Recommend sort/search for each situation with reasons.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Input size? Sorted? Need speed or memory? Stable? Worst vs average?
2. Small n , or the inner work is tiny, or you run it rarely.
3. Large sorted collection, many queries.
4. Binary search on unsorted roll numbers — may miss the target.

Long

1. Daily: selection/bubble or even by-hand; linear search. Archive: keep sorted, binary search (or a database index). State Big O and a precondition.